

Behelyettesíthetőség

Tekinstük az alábbi programot

```
#include <iostream>

class Person
{
public:
  Person(const std::string& name_, int age_):
    _name(name_),
    _age(age_)
  { }

  void print() const { std::cout << std::endl << _name << ", " << _age; }
protected:
  std::string _name;
  int _age;
};

class Employee: public Person
{
public:
  Employee(const std::string& name_, int age_, int employmentYear_):
    Person(name_, age_),
    _employmentYear(employmentYear_)
  { }

  void print() const { Person::print(); std::cout << ", " <<
_employmentYear << std::endl; }
private:
  int _employmentYear;
};

int main()
{
  Person person("Jack", 34);
  Employee employee("John", 28, 3);
```

```
    person.print(); // OK, meghívódik a Person::print()
    employee.print(); // OK, meghívódik az Employee::print()
}
```

A fenti példában nincs semmi meglepő. A tagfüggvényekhez objektumváltozókon keresztül férünk hozzá, így a változó típusa meghatároz egy tagfüggvényt és az hívódik meg.

Mutatók esetén azonban nem mindig ilyen egyértelmű a helyzet. Ilyenkor ugyanis két típus is szerepet játszik:

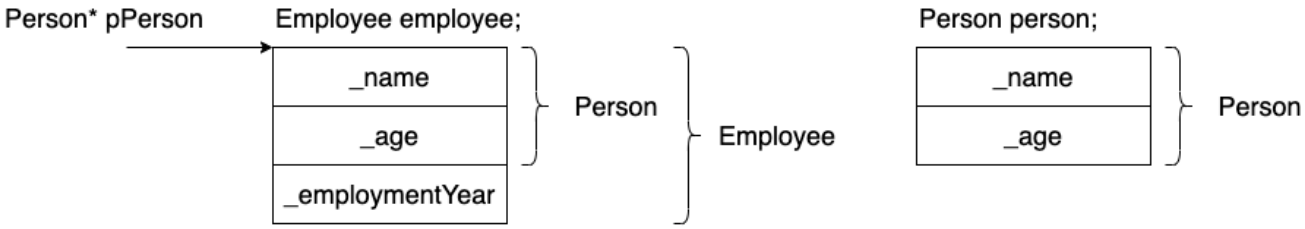
- a mutató típusa
- mutatott területen található változó típusa

Amikor ez a kettő különbözik, akkor a műveleteket a mutató típusa határozza meg.

Mutassunk rá egy **Employee** típusú objektumra egy **Person** típusú mutatóval.

```
Employee employee("John", 28, 3);
Person* pPerson = &employee;
```

Nézzük meg a két típus memóriabeli reprezentációját.



Jól látható, hogy az **Employee** típusú objektum eleje megegyezik egy **Person** típusú objektummal és a **pPerson** mutató valóban az **Employee** objektum **Person** típusú részére mutat. A mutatón keresztüli hozzáférés szempontjából az **Employee** osztály kompatibilis a **Person** osztállyal. Ez általában is igaz: ha egy osztály csak egy osztályból örököl, akkor mindig kompatibilis az ősosztállyal. Ekkor a **pPerson** mutatón keresztül minden olyan függvényhez hozzáférünk az **Employee** osztályban, amik megtalálhatóak a **Person**-ban is. A többihez viszont nincs hozzáférésünk a mutatón keresztül. Ezt nevezzük az objektum ledarabolódásának vagy **object slicing**-nak.

Érdekes kérdés lehet, hogy ebben az esetben a `print()` metódus hívása mit eredményez, ha a **pPerson** mutatón keresztül hívjuk meg. Mivel a műveletet a mutató típusa határozza meg és nem a mutatott memóriaterületen elhelyezkedő objektum típusa, így a `Person::print()` metódus fog meghívódni. Ha ehhez hozzávesszük, hogy mindezek nem csak mutatóra, hanem referenciára is igazak, kimondhatjuk a behelyettesíthetőség C++-beli megvalósításának lényegét:

Egy ősosztály típusú mutatónak mindig értékül adhatunk egy leszármazott típusú mutatót vagy leszármazott típusú objektum címét. Referencia esetén ez azt jelenti, hogy egy ősosztály típusú referenciát inicializálhatunk leszármazott típusú objektummal is. Az ősosztály típusú mutatón, illetve referencián keresztül csak az ősosztálybeli tagváltozók, illetve tagfüggvények érhetőek el.

Ennek következménye, hogy egy ősosztály típusú mutató esetében mi rejtőzik a mutató mögött. Ősosztály vagy származtatott osztály egy objektuma. Azt a jelenséget, hogy egy ősosztály típusú mutató mutathat bármelyik leszármazott típus példányára is, **polimorfizmusnak** nevezzük, a mutatót pedig **polimorfnak**.

Ha azt szeretnénk elérni, hogy ha egy ősosztály mutató mögött valamelyik leszármazott található, ne az ősosztály tagfüggvénye hívódjon meg, hanem a leszármazotté, akkor a `virtual` kulcsszót kell használnunk, melynek alkalmazásával az egyes tagfüggvények virtuálissá tehetők.

```
class Person
{
public:
  Person(const std::string& name_, int age_):
    _name(name_),
    _age(age_)
  { }

  virtual void print() { std::cout << _name << _age; }
protected:
  std::string _name;
  int _age;
};
```

A virtuális függvény (*virtual function*) azt jelenti, hogy:

- **ha az ősosztályban virtuálisnak deklarálunk egy tagfüggvényt, és**
- **a leszármazott osztályban létezik egy ugyanolyan nevű, argumentumlistájú és visszatérési értékű függvény, továbbá,**
- **ezt a függvényt egy leszármazott osztályra mutató, ősosztály típusú mutatón/referencián keresztül hívjuk meg,**

akkor a leszármazottbeli tagfüggvény fog meghívódni.

A `virtual` kulcsszót csak nem statikus tagfüggvényekre alkalmazhatjuk. Ha a leszármazott osztályban írunk egy ugyanolyan nevű, argumentumlistájú és visszatérési értékű függvényt, mint az ősosztály egy virtuális függvénye, akkor az ősosztály függvényét felülírjuk (**override**).

A leszármazott osztályban jelezni tudjuk, hogy egy ősosztálybeli függvényt írunk felül az `override` kulcsszó segítségével.

```
class Employee: public Person
{
public:
  Employee(const std::string& name_, int age_, int employmentYear_):
    Person(name_, age_),
    _employmentYear(employmentYear_)
  { }

  void print() override { Person::print(); std::cout << _employmentYear; }
private:
```

```
  int _employmentYear;
};
```

Megjegyzés: az `override` kulcsszót nem muszáj kiírni, viszont, ha kiírjuk figyeljünk oda, hogy csak olyan függvény esetében használhatjuk, amelyet az ősosztályban virtuálisnak jelöltünk.

Abban az esetben, ha az ősosztálybeli függvény annyira általános lenne, hogy nem tudunk neki viselkedést definiálni, viszont mégis kell hogy szerepeljen az ősosztályban, érdemes **tisztán virtuális** (*pure virtual*) függvényként megadni.

```
class Person
{
public:
  Person(const std::string& name_, int age_):
    _name(name_),
    _age(age_)
  { }

  virtual void print() = 0;
protected:
  std::string _name;
  int _age;
};
```

Ilyenkor a függvénytörzs helyett egy `= 0`-át írunk. Ezzel azt jelezzük a fordítónak, hogy nem kívánunk megadni függvénytörzset csak a leszármazott osztályban. Ha ezek után szeretnénk példányosítani a **Person** osztályt, és meghívnánk a `Person::print()` tagfüggvényt, a fordító bajban lenne, hiszen nem adtunk meg törzset a függvénynek, ezért a **Person** osztályt nem lehet példányosítani.

A legalább egy tisztán virtuális függvényt tartalmzató osztályt **absztrakt osztálynak** (*abstract class*) nevezzük. Az absztrakt osztályokat nem lehet példányosítani.

Ha egyszer egy függvényt virtuálisnak deklarálunk, akkor az összes leszármazottbeli osztályban virtuális lesz. A virtuális lánc NEM megszakítható!

A csak tisztán virtuális függvényeket tartalmazó osztályt **interfésznek** (*interface*) nevezzük. Ha öröklünk egy ilyen vagy egy absztrakt osztályból, biztosítanunk kell a fordítót, hogy a tisztán virtuális tagfüggvényekhez működést rendelünk.

Virtuális destruktor

Tekintsük az alábbi programot:

```
#include <iostream>

class Base
{
public:
```

```
  Base() { std::cout << "Base ctor" << '\n'; }
  ~Base() { std::cout << "Base dtor" << '\n'; }
};

class Derived : public Base
{
public:
  Derived() { std::cout << "Derived ctor" << '\n'; }
  ~Derived() { std::cout << "Derived dtor" << '\n'; }
};

int main()
{
  Base* p = new Derived();
  delete p;
}
```

Mi ezzel a probléma? Kimenet:

> Base ctor
>
> Derived ctor
>
> Base dtor

A származtatott osztály destruktora nem hívódott meg, ami abban az esetben, ha dinamikus adattagokat használtunk volna benne memóriaszivárgást jelentene. Erre a problémára nyújt megoldást, ha a destruktort is virtuálissá tesszük.

```
#include <iostream>

class Base
{
public:
  Base() { std::cout << "Base ctor" << '\n'; }
  virtual ~Base() { std::cout << "Base dtor" << '\n'; }
};
```

Kimenet:

> Base ctor
>
> Derived ctor
>
> Derived dtor
>
> Base dtor

Ha az alaposztály destruktora virtuális, akkor minden ebből származtatott osztály destruktora is virtuális lesz. Ezáltal biztosak lehetünk abban, hogy a megfelelő destruktor hívódik meg, amikor az objektum

megszűnik, még akkor is, ha valamelyik alaposztály típusú mutatóval vagy referenciával hivatkozunk a leszármazott osztály példányára.

Virtuális függvények megvalósítása

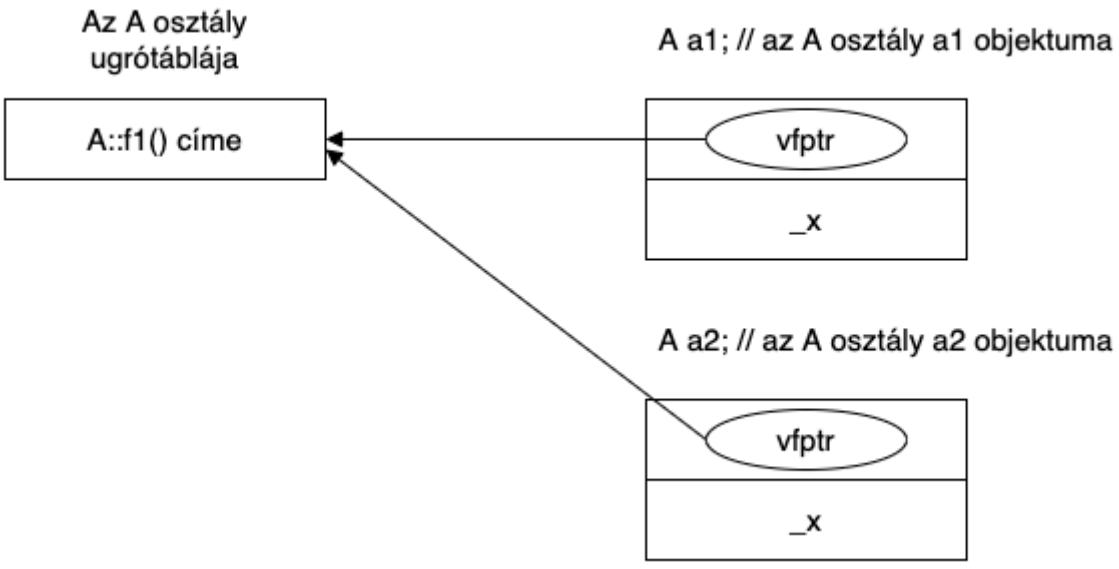
A nem virtuális függvények esetében az a cím, amelyre a függvény hívásakor ugrani kell, fordítási időben határozódik meg. Egy virtuális függvény mutatón, illetve referencián keresztüli hívásakor azonban az a függvénycím, amelyre ugrani kell, csak futási időben dől el, a fordító nem "égeti be" egyik függvény címét se. A fordítók ilyen esetekben indirekciót használnak. Minden olyan osztály, amelynek van legalább egy virtuális függvénye (akár definiált, akár örökölt), rendelkezik egy táblázattal a virtuális függvények ugrási címeivel. Ezt szokás virtuális ugrótáblának nevezni. A tábla annyi címet tartalmaz, ahány virtuális függvénye az adott osztálynak van.

Az osztály minden objektumának van egy mutatója az osztályának virtuális ugrótáblájához. A mutatóra a **vfptr** (*virtual function table pointer*) a leggyakrabban alkalmazott jelölés. A mutató minden objektumban ténylegesen jelen van és helyet foglal. Tekintsük az alábbi kódrészletet:

```
class A
{
public:
  virtual void f1() {}
private:
  int _x:
};

int main()
{
  A a1;
  A a2;
}
```

A memóriakép a következő lesz:



Virtuális függvények öröklődés esetén:

